

République Tunisienne
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
École Supérieure Privée d'Ingénierie et de Technologie — TEK-UP

Conception et développement d'une plateforme d'inventaire logistique multi-canal

Rapport de Projet de Fin d'Études — Lot 5

Pages finales (listes, résumé, conclusion, bibliographie)

Réalisé par : **Harrabi Yosri**

Spécialité : Génie Logiciel et Systèmes d'Information

Encadrant professionnel : M. Ferchichi Marwen — VECTORYS Tunisie

Encadrante académique : Mme Zrelli Siwar — TEK-UP

Année universitaire 2025 — 2026

Table des matières — Lot 5

Liste des abréviations

Liste des figures

Liste des tableaux

Résumé (FR) / Abstract (EN) — version consolidée

Conclusion générale

Perspectives

Bibliographie

Liste des abréviations

Sigle	Signification
API	Application Programming Interface
C1 / C2	Premier / second passage de comptage d'inventaire
CORS	Cross-Origin Resource Sharing
CQRS	Command Query Responsibility Segregation
DTO	Data Transfer Object
E2E	End-to-End (bout en bout)
EF Core	Entity Framework Core
ERP	Enterprise Resource Planning
HTTP / HTTPS	Hypertext Transfer Protocol (Secure)
IIS	Internet Information Services
JWT	JSON Web Token
KPI	Key Performance Indicator (indicateur de performance)
LAN	Local Area Network
ORM	Object-Relational Mapping
PDA	Personal Digital Assistant (terminal mobile)
PFE	Projet de Fin d'Études
REST	Representational State Transfer
RF / RNF	Exigence Fonctionnelle / Non Fonctionnelle
SGBD	Système de Gestion de Base de Données
SQL	Structured Query Language
SSMS	SQL Server Management Studio
TTL	Time To Live (durée de validité du jeton)
UC	Use Case (cas d'utilisation)
UML	Unified Modeling Language
US	Unité de Stock (code scanné)
WMS	Warehouse Management System

Liste des figures

Chapitre 1

Figure 1.1 — Architecture three-tier de la plateforme Vectorys Inventaire
Figure 1.2 — Flux métier principal du cycle d'inventaire Vectorys
Figure 1.3 — Cadre Scrum et répartition des rôles dans le projet
Figure 1.4 — Frise chronologique des sprints 0 à 9 (version finale)

Chapitre 2

Figure 2.1 — Écran de création d'inventaire et import du stock théorique
Figure 2.2 — Interface agent : zones, scan en boucle et progression US
Figure 2.3 — Dashboard KPI (précision, performance, qualité, clients, WMS)
Figure 2.4 — Diagramme de cas d'utilisation détaillé

Chapitre 3

Figure 3.1 — Architecture logique three-tier
Figure 3.2 — Diagramme de classes global (vue métier)
Figure 3.3 — Diagramme de séquence : authentification
Figure 3.4 — Interface d'authentification Angular
Figure 3.5 — Interface d'authentification Flutter
Figure 3.6 — Diagramme de séquence : création d'inventaire et génération C1
Figure 3.7 — Formulaire de création / configuration d'inventaire
Figure 3.8 — Vue conceptuelle de l'import Excel vers le modèle de données
Figure 3.9 — Import du stock théorique et validation CodeAgent
Figure 3.10 — Diagramme de séquence : scan, quota et notification
Figure 3.11 — Classes concernées par le comptage terrain
Figure 3.12 — Emplacements assignés (mobile)
Figure 3.13 — Scan en boucle et progression par unité de stock
Figure 3.14 — Diagramme de séquence : détection et Branche A
Figure 3.15 — Liste des écarts et choix de stratégie
Figure 3.16 — Diagramme de séquence : consultation KPI
Figure 3.17 — Suivi d'avancement en temps réel
Figure 3.18 — Tableau de bord KPI

Chapitre 4

Figure 4.1 — Chaîne de développement locale (clients → API → SQL Server)
Figure 4.2 — Couches de la solution .NET réalisées
Figure 4.3 — Portail web : authentification superviseur

Figure 4.4 — Application mobile : authentification agent
Figure 4.5 — Création d'inventaire et préparation de l'import
Figure 4.6 — Suivi d'avancement temps réel (canal web)
Figure 4.7 — Gestion des écarts (branche A)
Figure 4.8 — Tableau de bord KPI réalisé
Figure 4.9 — Liste des emplacements assignés (mobile)
Figure 4.10 — Boucle de scan et progression par US
Figure 4.11 — Démarrage du comptage sur un emplacement
Figure 4.12 — Progression du scan pendant le passage C1
Figure 4.13 — Chaîne de synchronisation offline (Flutter)
Figure 4.14 — Enchaînement des sprints de réalisation
Figure 4.15 — Topologie de déploiement sur serveur local
Figure 4.16 — API publiée sur le serveur local (Swagger)
Figure 4.17 — Portail superviseur déployé en local *(à compléter)*
Figure 4.18 — Application mobile configurée pour le serveur local *(à compléter)*

Chapitre 5

Figure 5.1 — Niveaux de test privilégiés
Figure 5.2 — Point d'entrée des tests API : Swagger
Figure 5.3 — Jeu de données Excel / rapport d'assignation *(à compléter)*
Figure 5.4 — Test d'intégration API (Postman) *(à compléter)*
Figure 5.5 — Étape superviseur : création avant import de test
Figure 5.6 — Étape agent : liste filtrée des emplacements
Figure 5.7 — Étape agent : progression du scan et quotas
Figure 5.8 — Vérification BDD après scénario E2E *(à compléter)*
Figure 5.9 — Chaîne vérifiée lors des tests offline
Figure 5.10 — Contrôle superviseur : suivi d'avancement
Figure 5.11 — Contrôle superviseur : écarts détectés
Figure 5.12 — Contrôle superviseur : indicateurs KPI
Figure 5.13 — Séance de validation / démonstration *(optionnel)*

Liste des tableaux

Chapitre 1

Tableau 1.1 — Stack technologique de la solution
Tableau 1.2 — Découpage des sprints 0 à 9

Chapitre 2

Tableau 2.1 — Acteurs du système
Tableau 2.2 — Exigences d'authentification (RF-01 à RF-04)
Tableau 2.3 — Exigences inventaires et passages
Tableau 2.4 — Exigences d'import et d'assignation
Tableau 2.5 — Exigences de comptage terrain
Tableau 2.6 — Exigences de gestion des écarts
Tableau 2.7 — Exigences KPI et temps réel
Tableau 2.8 — Exigences non fonctionnelles (RNF)
Tableau 2.9 — Catalogue des cas d'utilisation
Tableau 2.10 — Backlog produit (user stories par sprint)
Tableau 2.11 — Matrice de traçabilité exigences / composants

Chapitre 3

Tableau 3.1 — Énumérations du domaine métier
Tableau 3.2 — Cas d'utilisation détaillés (authentification … pilotage)
Tableau 3.3 — Vue synthétique des endpoints API

Chapitre 4

Tableau 4.1 — Environnement de développement et outillage
Tableau 4.2 — Migrations EF Core
Tableau 4.3 — Endpoints métier et hub SignalR
Tableau 4.4 — Livrables techniques par sprint
Tableau 4.5 — Architecture de déploiement sur serveur local
Tableau 4.6 — Difficultés rencontrées et solutions

Chapitre 5

Tableau 5.1 — Niveaux de test et outils
Tableau 5.2 — Configuration de l'environnement de test
Tableau 5.3 — Jeu de données Excel de référence
Tableau 5.4 — Cas de test API (T-API-01 à T-API-13)

Tableau 5.5 — Critères d'acceptation du scénario E2E
Tableau 5.6 — Cas de test offline
Tableau 5.7 — Matrice de couverture des exigences
Tableau 5.8 — Vérifications non fonctionnelles

Résumé

Ce projet de fin d'études, réalisé au sein de **VECTORYS Tunisie**, porte sur la conception et le développement de **Vectorys Inventaire**, une plateforme d'inventaire logistique multi-canal. L'objectif est de digitaliser le cycle complet de l'inventaire physique en entrepôt : préparation (création de mission, import du stock théorique depuis Excel, assignation stricte des agents), exécution terrain (scan des unités de stock, quotas, clôture automatique des emplacements, mode hors ligne), traitement des écarts (détection MANQUANT/SURPLUS, recomptage ciblé) et pilotage (suivi temps réel SignalR, tableaux de bord KPI).

Sur le plan technique, la solution repose sur une architecture three-tier : une API REST **ASP.NET Core 9** (Clean Architecture, CQRS/MediatR, FluentValidation, JWT, Entity Framework Core, ClosedXML), une application web **Angular 19** destinée au superviseur, et une application mobile **Flutter** pour les agents, adossées à **Microsoft SQL Server** et à une base SQLite locale pour la résilience hors connexion. Le projet a été mené selon la méthode agile Scrum (sprints 0 à 9). La validation a combiné tests d'intégration API (Swagger, Postman), scénarios bout-en-bout import/comptage et contrôles en base de données. Le déploiement retenu pour l'exploitation interne est un **serveur local** sur le réseau d'entreprise.

Mots-clés : inventaire logistique, multi-canal, ASP.NET Core, Angular, Flutter, SQL Server, SignalR, Scrum, KPI, entrepôt, hors ligne.

Abstract

This graduation project, carried out at **VECTORYS Tunisia**, focuses on the design and development of **Vectorys Inventaire**, a multi-channel logistics inventory platform. The goal is to digitize the full physical warehouse inventory cycle: preparation (mission creation, Excel import of theoretical stock, strict agent assignment), field execution (unit scanning, quotas, automatic location closure, offline mode), discrepancy handling (shortage/surplus detection, targeted recount), and supervision (real-time SignalR monitoring, KPI dashboards).

Technically, the solution follows a three-tier architecture: an **ASP.NET Core 9** REST API (Clean Architecture, CQRS/MediatR, FluentValidation, JWT, Entity Framework Core, ClosedXML), an **Angular 19** web portal for supervisors, and a **Flutter** mobile application for agents, backed by **Microsoft SQL Server** and a local SQLite store for offline

resilience. The work was organized with Scrum (sprints 0–9). Validation combined API integration tests (Swagger, Postman), end-to-end import/counting scenarios, and database checks. The deployment model chosen for internal use is a **local on-premises server** on the corporate LAN.

Keywords: logistics inventory, multi-channel, ASP.NET Core, Angular, Flutter, SQL Server, SignalR, Scrum, KPI, warehouse, offline.

Note : une version courte du résumé/abstract figure déjà dans le Lot 1 (pages liminaires). Le texte ci-dessus constitue la version consolidée, enrichie après la rédaction des chapitres 3 à 5 (conception, réalisation, tests et déploiement local).

Conclusion générale

Au terme de ce projet de fin d'études, la problématique posée au chapitre 1 — concevoir une plateforme d'inventaire multi-canal fiable, capable de synchroniser le comptage terrain et le pilotage superviseur, y compris hors connexion — a trouvé une réponse concrète à travers **Vectorys Inventaire**.

Le cheminement du rapport a permis de passer d'une analyse métier (acteurs, RF/RNF, cas d'utilisation, backlog Scrum) à une conception technique structurée (architecture three-tier, modèle de classes, séquences par fonctionnalité), puis à une réalisation multi-canal (API Clean Architecture, portail Angular, application Flutter offline) et enfin à une validation par scénarios reproductibles. La plateforme couvre désormais le cœur du cycle inventaire VECTORYS : import Excel, assignation CodeAgent, scan à quotas, auto-terminaison, sync batch, détection d'écarts avec branche A, suivi temps réel et indicateurs KPI.

Sur le plan méthodologique, Scrum a favorisé des livraisons incrémentales et une priorisation guidée par la valeur terrain. Sur le plan organisationnel, le binôme encadrant professionnel / encadrante académique a ancré les choix techniques dans les contraintes réelles de l'entrepôt (zones couvertes partiellement, fichiers ERP hétérogènes, besoin de supervision immédiate).

Les limites identifiées — couverture encore partielle des tests unitaires automatisés, finalisation de certains workflows de clôture/export, captures de déploiement LAN encore à compléter — n'effacent pas le résultat principal : une plateforme opérationnelle, documentée et prête à être mise en service sur un **serveur local** du réseau VECTORYS.

Ce travail a également constitué une expérience professionnelle complète : compréhension d'un métier logistique 3PL, conception logicielle en Clean Architecture, développement multi-technologie et conduite de tests d'acceptation. Il ouvre naturellement sur les perspectives exposées ci-après.

Perspectives

Plusieurs axes d'évolution se dégagent à l'issue du stage :

1. **Mise en service sur serveur local.** Publier l'API et le portail sur la machine d'entreprise, configurer CORS/SignalR et générer l'APK avec `API_BASE_URL` pointant vers l'IP du serveur ; compléter alors les figures 4.17 et 4.18.
2. **Consolidation des workflows de fin d'inventaire.** Formaliser la clôture inventaire (statut *CLOTURE*), l'export Excel des résultats et, selon besoin terrain, le recomptage global (branche B).
3. **Automatisation des tests.** Enrichir la suite xUnit / Flutter Test autour des handlers critiques (quotas, écarts, import) pour sécuriser les régressions.
4. **Enrichissement analytique.** Affiner les KPI WMS/clients (référentiel client, valeur monétaire des écarts, occupation maximale) à partir de données ERP complémentarisées.
5. **Expérience terrain.** Intégrer retours opérateurs (ergonomie scan, alertes, impressions de rapports) et éventuellement le mode tablette pour les chefs d'équipe.
6. **Industrialisation.** Scripts de publication, secrets hors dépôt, journaling opérationnel et procédure de reprise après incident.

Ces perspectives marquent la transition entre un livrable académique réussi et un produit d'exploitation durable au sein de VECTORYS Tunisie.

Bibliographie

[1] Microsoft, *ASP.NET Core documentation*, available at: https://learn.microsoft.com/aspnet/core/ (consulted 2026).

[2] Microsoft, *Entity Framework Core documentation*, available at: https://learn.microsoft.com/ef/core/ (consulted 2026).

[3] Microsoft, *ASP.NET Core SignalR*, available at: https://learn.microsoft.com/aspnet/core/signalr/ (consulted 2026).

[4] Angular Team, *Angular Developers Documentation*, available at: https://angular.dev/ (consulted 2026).

[5] Flutter Team / Google, *Flutter documentation*, available at: https://docs.flutter.dev/ (consulted 2026).

[6] OASIS / OpenAPI Initiative, *OpenAPI Specification*, available at: https://swagger.io/specification/ (consulted 2026).

[7] Schwaber, K. & Sutherland, J., *The Scrum Guide*, Scrum.org, latest official edition, available at: https://scrumguides.org/ (consulted 2026).

[8] Martin, R. C., *Clean Architecture: A Craftsman's Guide to Software Structure and Design*, Prentice Hall, 2017.

[9] Fowler, M., *Patterns of Enterprise Application Architecture*, Addison-Wesley, 2002 (CQRS / layered design — foundation concepts).

[10] Object Management Group, *Unified Modeling Language (UML) specification*, available at: https://www.omg.org/spec/UML/ (consulted 2026).

[11] ClosedXML contributors, *ClosedXML documentation* (Excel for .NET), available at: https://closedxml.readthedocs.io/ (consulted 2026).

[12] IETF, *JSON Web Token (JWT) — RFC 7519*, available at: https://datatracker.ietf.org/doc/html/rfc7519 (consulted 2026).

[13] VECTORYS Tunisie, documentation interne et besoins métier inventaire (notes de stage, entretiens encadrant, 2025–2026).

[14] TEK-UP, *Template et consignes de rédaction du rapport de Projet de Fin d'Études*, École Supérieure Privée d'Ingénierie et de Technologie, année universitaire 2025–2026.

— Fin du Lot 5 / pages finales du rapport —